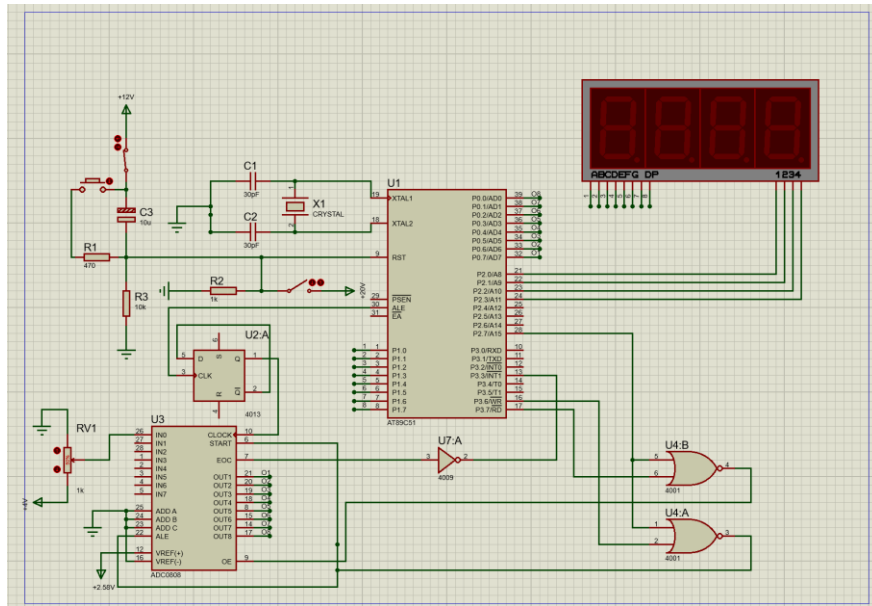


作业三

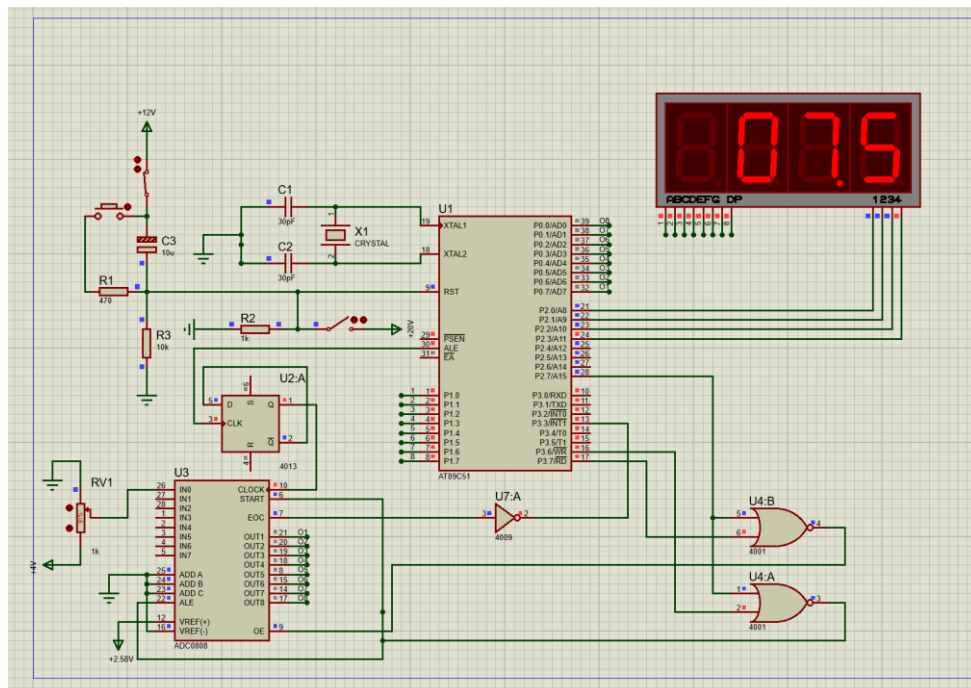
1. 复现实验四

1.1 数码管

硬件连线如下:



软件由 PPT 中给出, 选取滑动变阻器的两个状态截图, 得到结果如下



3. 执行特定程序

程序如下和结果如下：

ORG 0000H

LJMP START

ORG 0040H

START: MOV 23H, #30H ; 内存地址 d:0x0023 变为 30H

MOV 12H, #34H ; d:0x0012 变为 34H

MOV R0, #23H ; 寄存器 R0 内容为 23H

MOV R7, 12H ; 寄存器 R7 内容变为内存地址 d:0x0012 处的值，即 34H

MOV R1, #12H ; 寄存器 R1 内容变为 12H

； 结果如下：

Register	Value
r0	0x23
r1	0x12
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x34

Memory 1	
Address: d:0x0012	
D:0x12:	34 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 30 00 00 00 00 00 00 00
D:0x2B:	00 00
D:0x44:	00 00
D:0x5D:	00 00

MOV A, @R0 ; 将内存地址为 R0 的值的内存单元的值放入 A 中

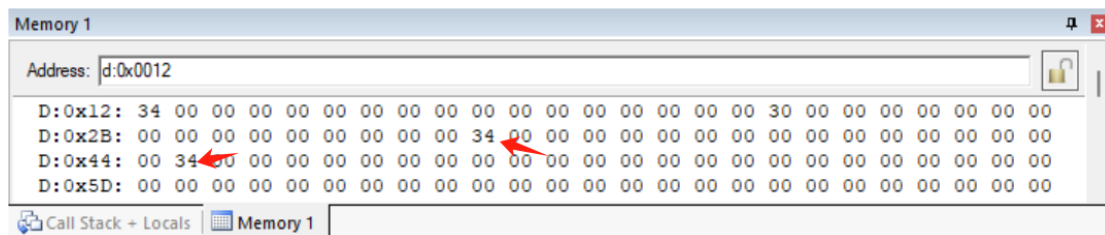
； 即 A=(23H)=34H

MOV 34H, @R1 ; 内存单元 34H 的值变为 34H，即(34H)=(12H)=34H

MOV 45H, 34H ; 内存单元 45H 的值变为 34H，即(45H)=(34H)=34H

； 结果如下

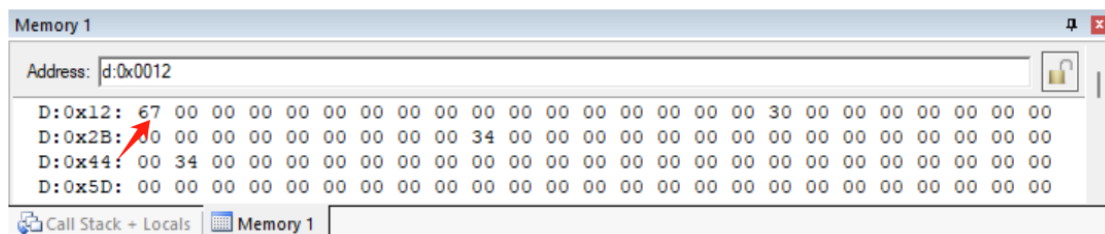
Sys	
a	0x30
b	0x00
sp	0x07
sp_max	0x07
dptr	0x0000
PC \$	C:0x0052
states	15
sec	0.000007!
psw	0x00



MOV DPTR, #6712H ; DPTR 寄存器的值变为 6712H
 MOV 12H, DPH ; 内存单元 12H 的值变为 DPTR 的高字节, 即(12H)=67H
 MOV R0, DPL ; 寄存器 R0 的值变为 DPTR 的低字节, 即 R0=12H
 MOV A, @R0 ; 寄存器 A 的值变为 R0 的值指向的地址的值
 ; 即 A=(R0)=(12H)=67H

; 结果如下

Register	Value
Regs	
r0	0x12
r1	0x12
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x34
Sys	
a	0x67
b	0x00
sp	0x07
sp_max	0x07
dptr	0x6712
PC \$	C:0x005B
states	22
sec	0.0000111
psw	0x01



LOOP: NOP
 JMP LOOP
 END

4. KEIL 和 PROTEUS 使用

4.1 KEIL 仿真带进位的减法

程序如下：

```
ORG 0000H
```

```
    LJMP START
```

```
ORG 0040H
```

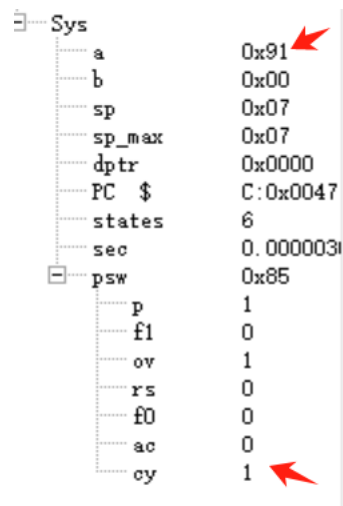
```
    START: CLR C           ; 清空 PSW 的 CY
```

```
          MOV A, #34H
```

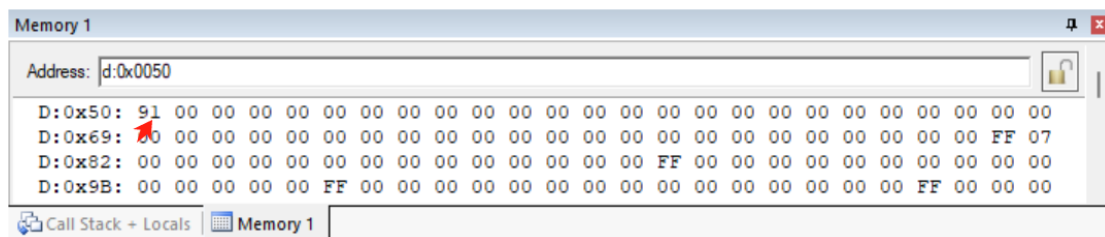
```
          SUBB A, #0A3H
```

```
          MOV 50H, A       ; 至此 A=91H, (50H)=91H, CY=1
```

； 结果如下



Sys	
a	0x91
b	0x00
sp	0x07
sp_max	0x07
dptr	0x0000
PC \$	C:0x0047
states	6
sec	0.0000031
psw	0x85
p	1
f1	0
ov	1
rs	0
f0	0
ac	0
cy	1



Memory 1	
Address: d:0x0050	
D:0x50:	91 00
D:0x69:	00 00
D:0x82:	00 00
D:0x9B:	00 00 00 00 00 00 FF 00

```
    MOV A, #12H
```

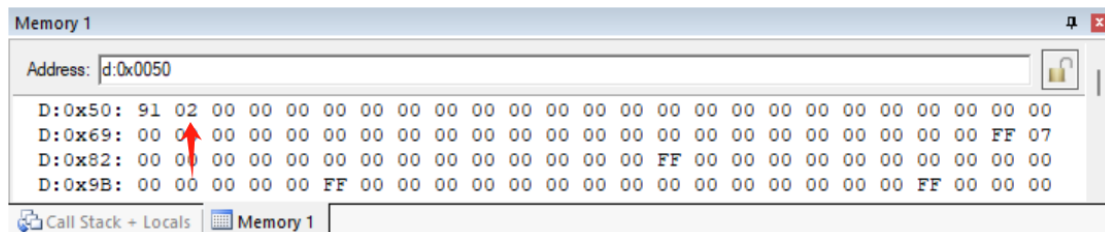
```
    SUBB A, #0FH
```

```
    MOV 51H, A
```

； 至此 A=2H，因为 CY=1，实际是 12H-0FH-01H=2H

； (51H)=02H, CY 清空，但是这里发生了半借位，AC=1

； 结果如下



程序如下：

```
ORG 0000H
```

```
    LJMP START
```

```
ORG 0040H
```

```
START: MOV P1, #0FFH
```

```
      MOV P3, #0FFH
```

```
L1:    JNB P3.2, L2
```

```
      JNB P3.3, L3
```

```
      LJMP L1
```

```
L2:    MOV    P1, #00H
```

```
      LJMP L1
```

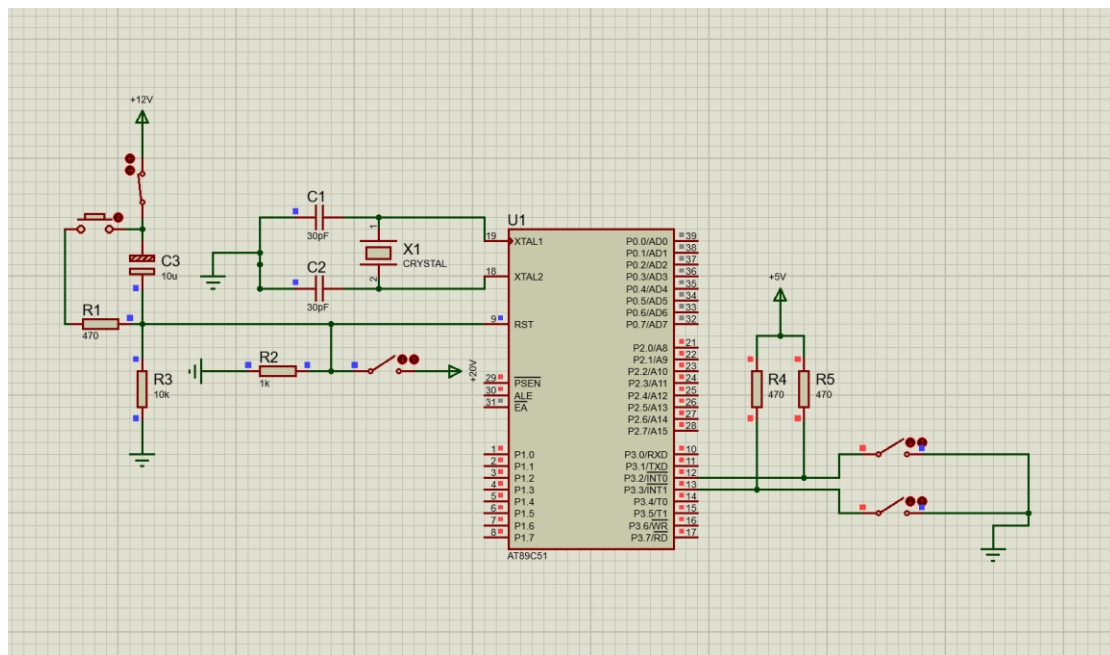
```
L3:    MOV P1, #0FFH
```

```
      LJMP L1
```

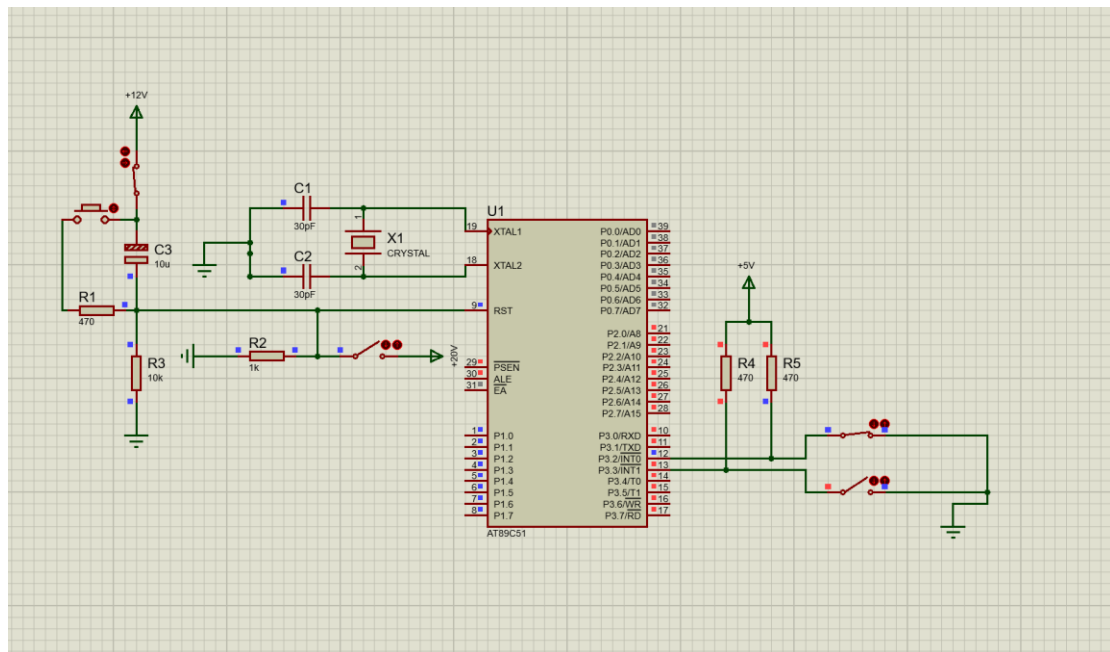
```
END
```

仿真结果如下：

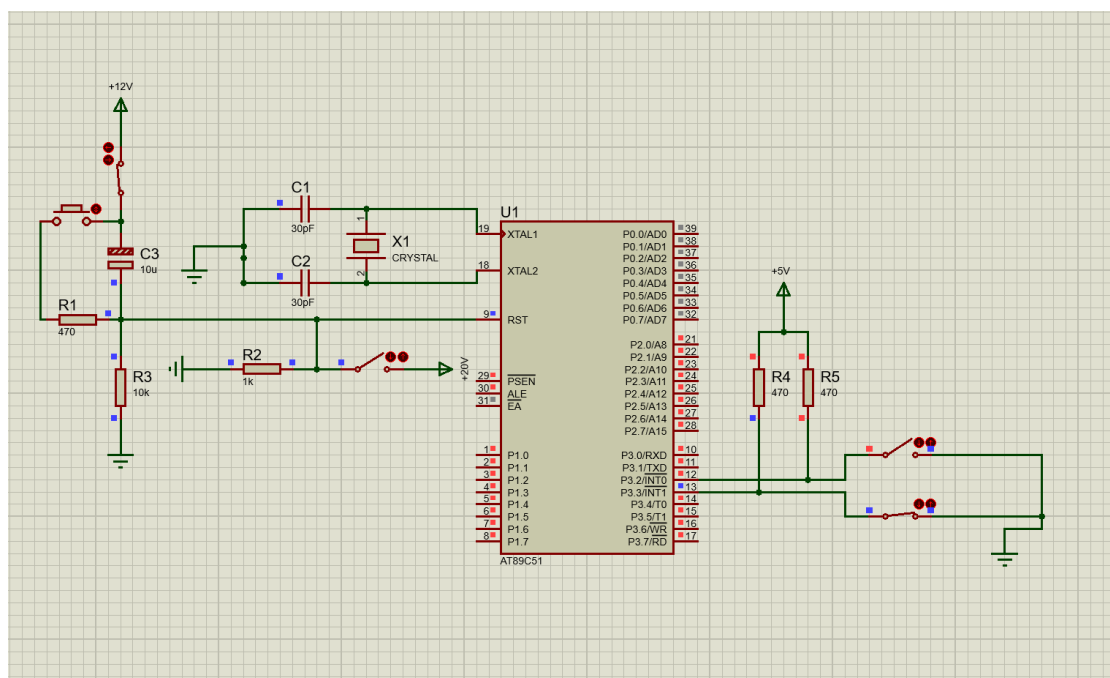
刚开始，按钮都没按下时，输出全 1，符合程序预期



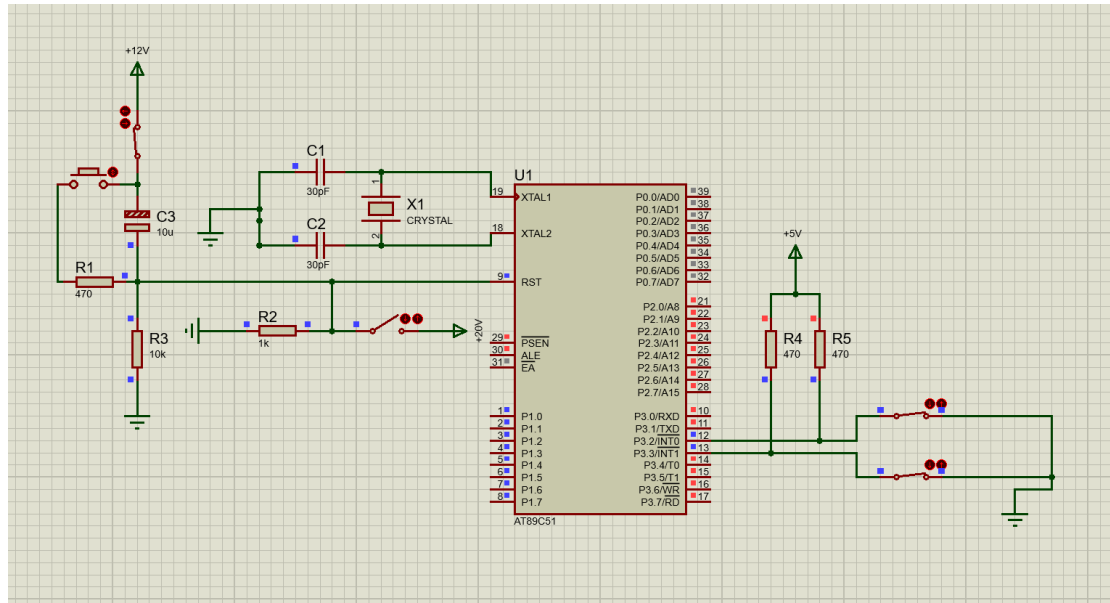
按下 P3.2 对应的按钮，跳转到 L2，输出全 0



松开 P3.2 对应的按钮，再按下 P3.3 对应的按钮，输出全 1



当 P3.2 保持按下时，检测不到 P3.3 的状态，原因在于程序中 L2 在 L3 之上执行：
当 P3.2 按下，程序跳转执行 L2，P1 全 0，返回 L1；此时又检测到 L2 成立，重复上面步骤
这样永远无法检测 P3.3 的状态



实验结果支持上述的观点